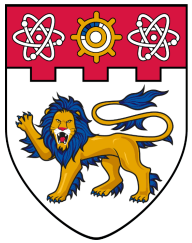
Artificial Neural Networks

Fedor Duzhin

class 5



NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

1 Intro

2 Gradient descent

3 Logistic regression

4 Feedforward neural networks

5 Training neural networks

6 Conclusion

7 Summary

Section 1

Intro

Revision of previously learned material

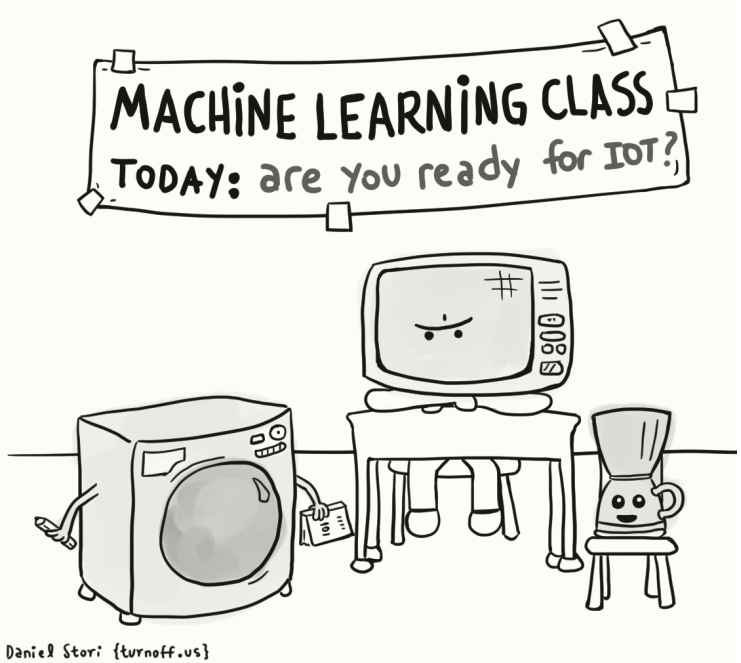
We have learned two simple parameteric models for supervised learning — linear regression (for regression problems) and logistic regression (for classification problems). Both are trained by minimizing a suitable loss function.

We have also seen l_1-regularization, also called LASSO — a method for penalizing a model for being too complex to prevent overfitting.

LASSO involves a hyperparameter λ that governs the size of the penalty. The optimal value λ is usually found by cross-validation.

Learning outcomes of today's class

1. Know how to train a feedforward neural network model
2. Understand the basic idea of gradient descent, its pitfalls, and methods to improve plain-vanilla gradient descent.
3. Choose an appropriate loss function for a neural network.
4. Regularize deep learning models

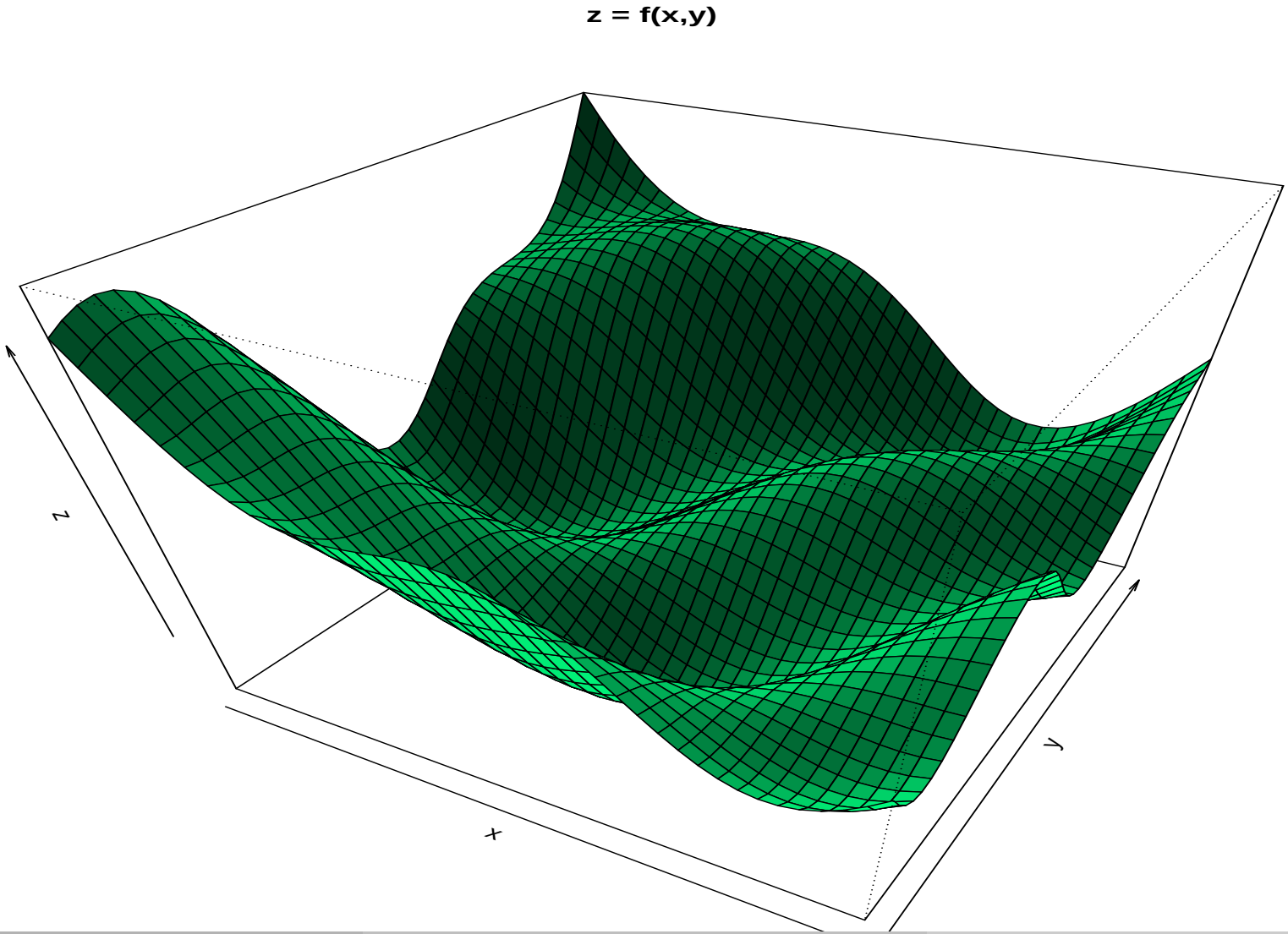


Section 2

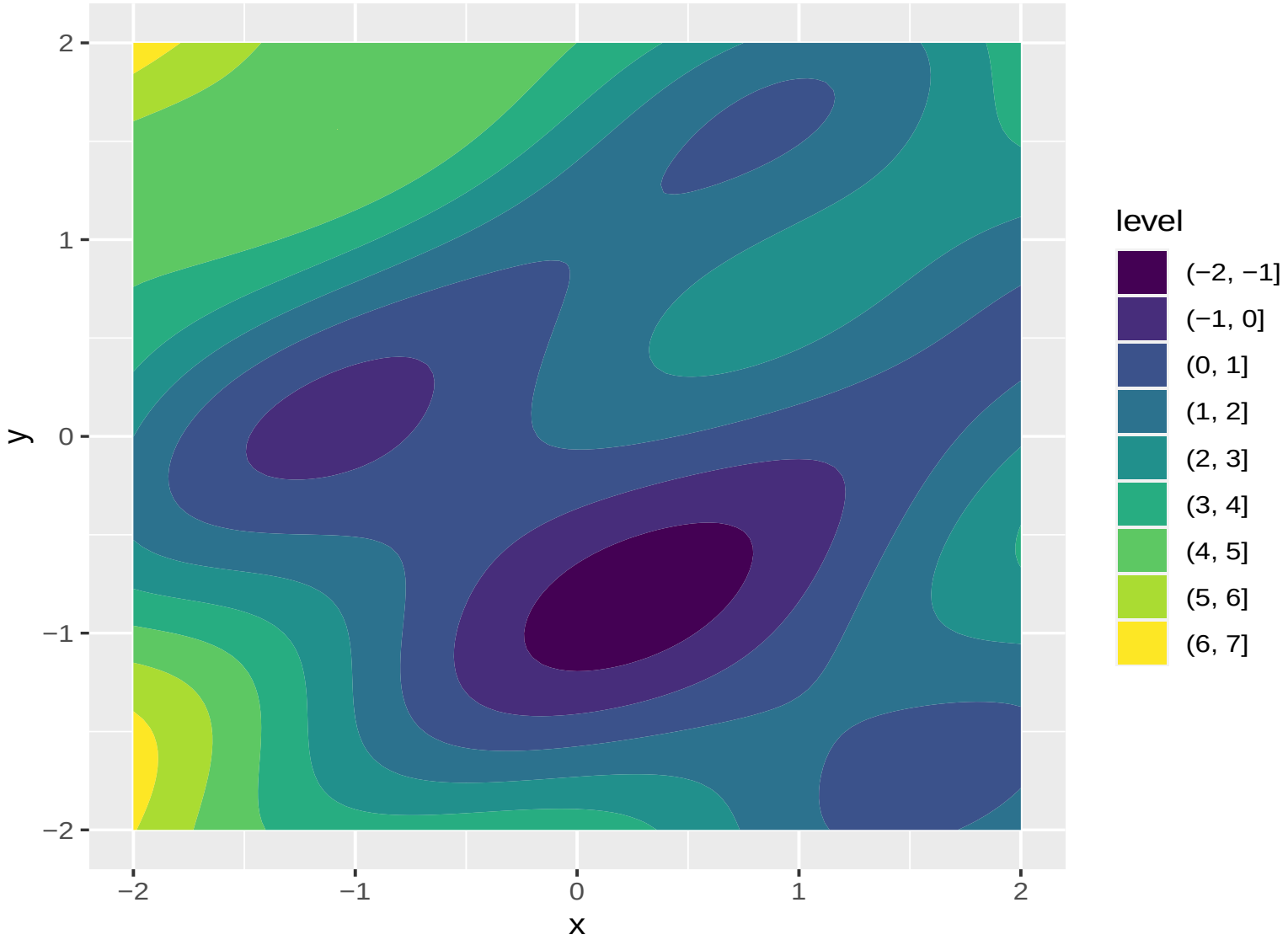
Gradient descent

Calculus preliminaries

The graph of a function $z = f(x, y)$ is a surface in the 3D space.



Another way to visualize a function $z = f(x, y)$ is a contour plot:



The *gradient* of a function $f(x, y)$ is the vector

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

The gradient is orthogonal to level curves of the function and points at the direction in which the function increases at the fastest rate.

Gradient descent (*learning rate* $\alpha > 0$ and *initial point* (x_0, y_0) are hyperparameters):

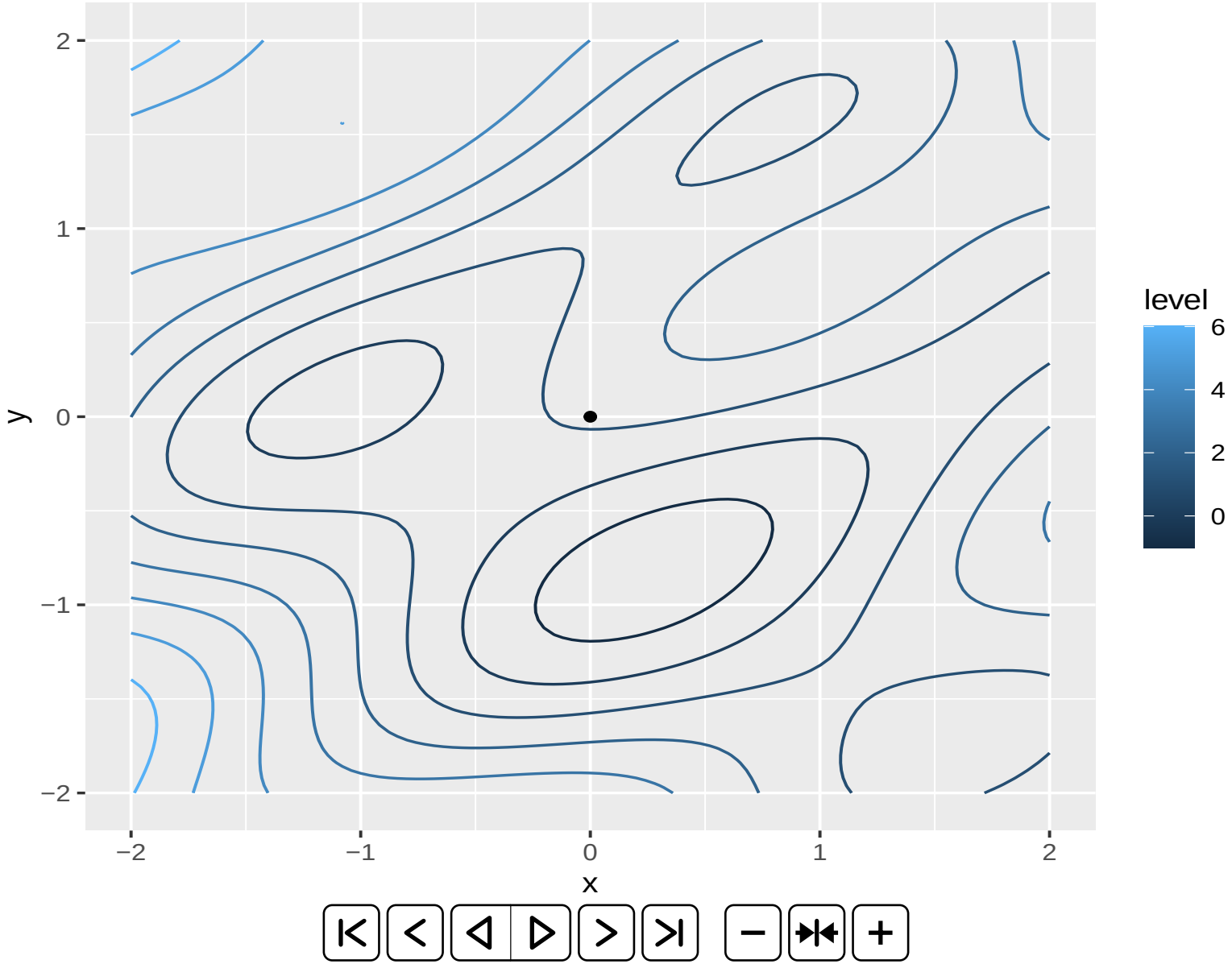
- Begin at some initial point (x_0, y_0) and find the gradient vector $\nabla f(x_0, y_0)$.
- Make a small step in the direction $-\nabla f$, i.e.,

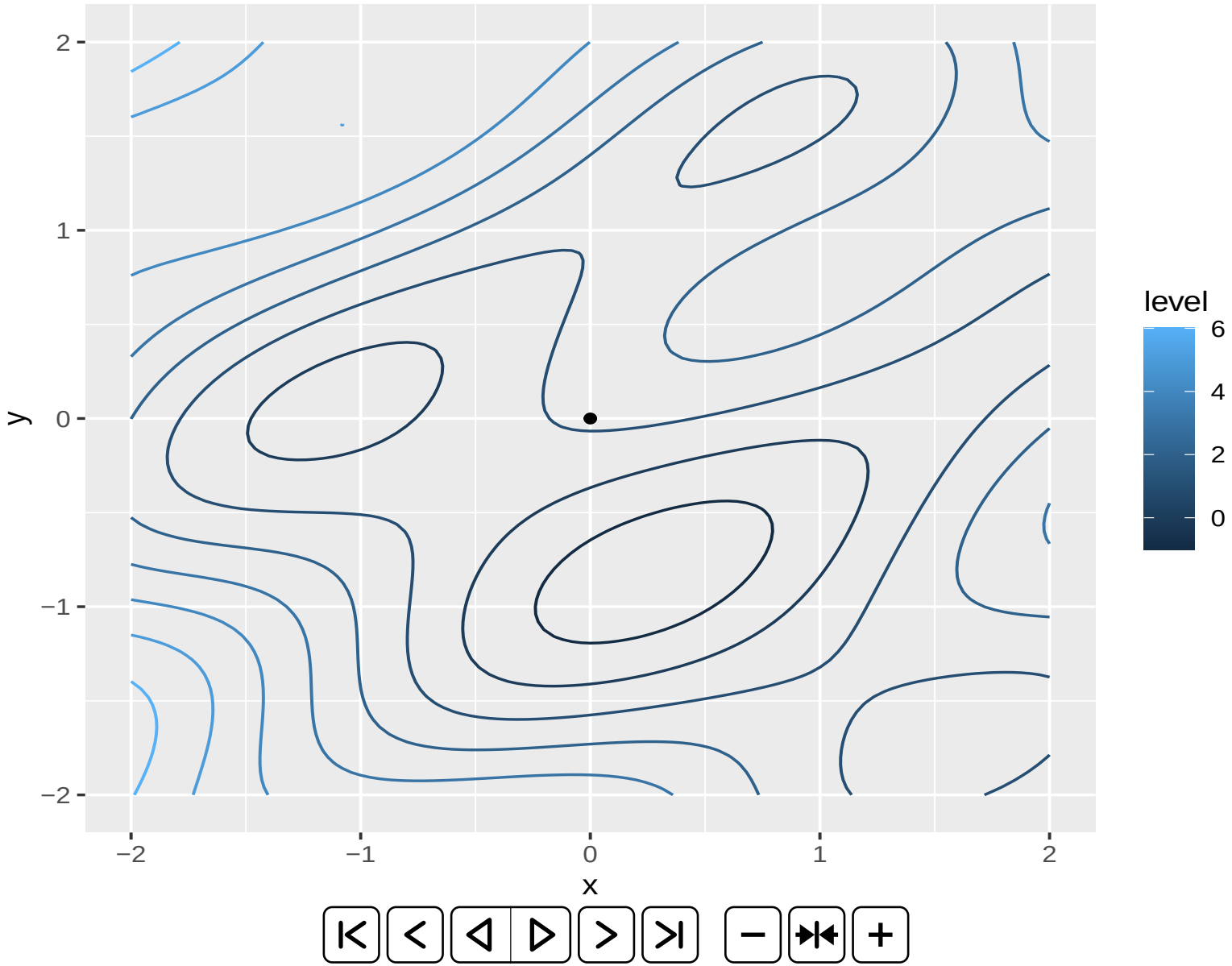
$$(x_1, y_1) = (x_0, y_0) - \alpha \nabla f(x_0, y_0)$$

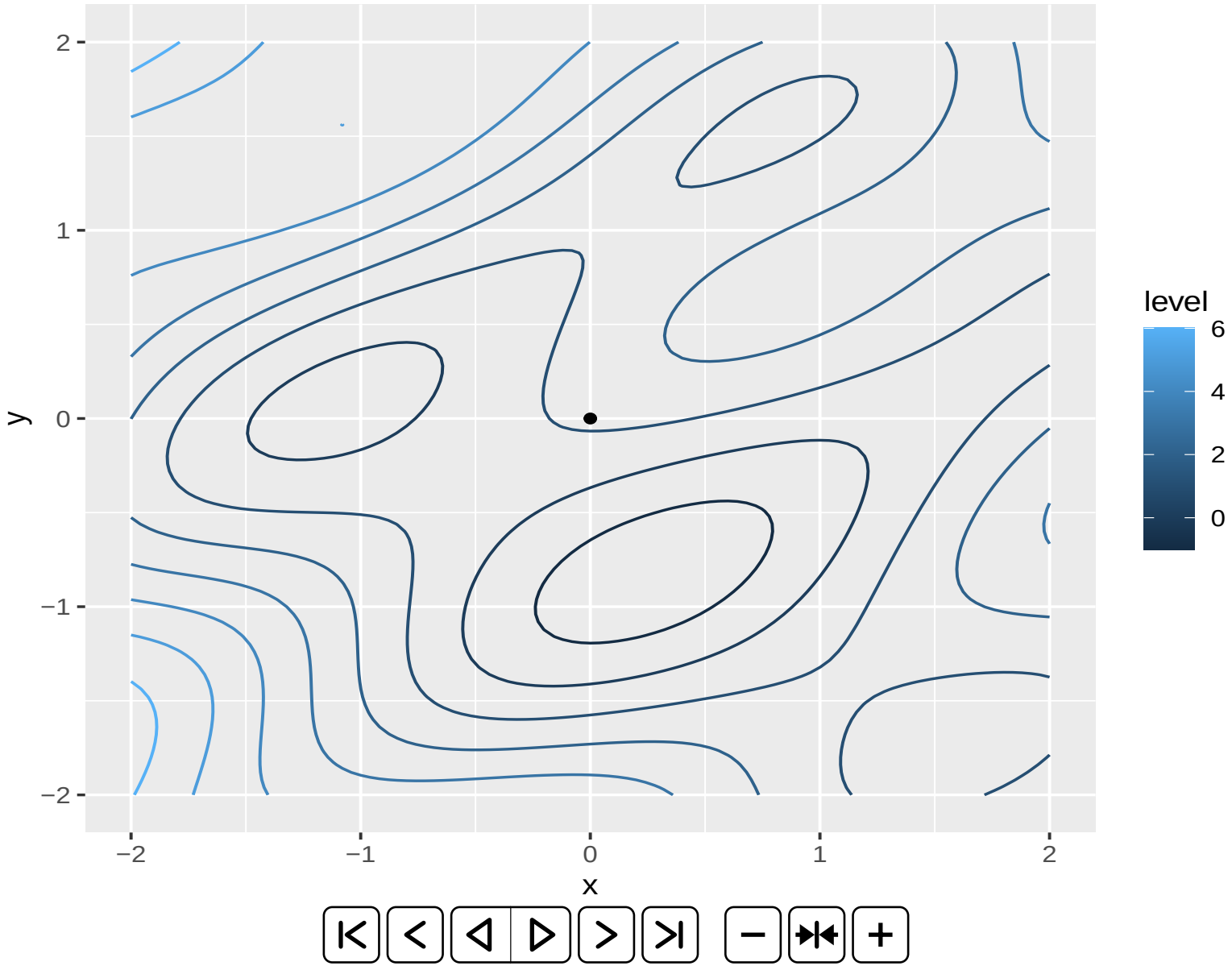
- Make a small step in the direction $-\nabla f$ again:

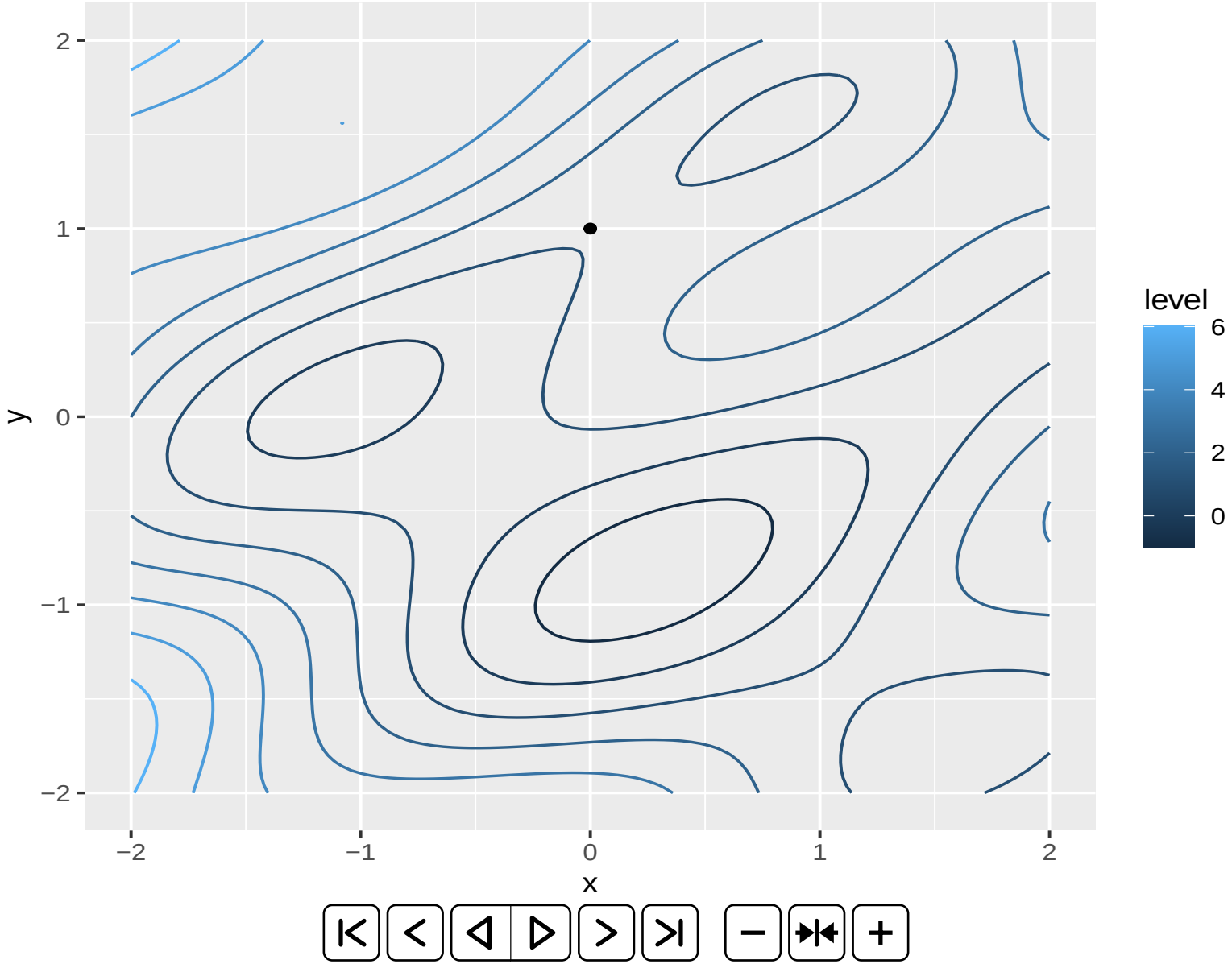
$$(x_2, y_2) = (x_1, y_1) - \alpha \nabla f(x_1, y_1)$$

- Etc. Either stop after a fixed number of steps or when $\nabla f \approx 0$.


2
1
0
−1
−2
y
−2
−1
0
1
2
x
level
6
4
2
0


2
1
0
−1
−2
y
−2
−1
0
1
2
x
level
6
4
2
0


2
1
0
−1
−2
y
−2
−1
0
1
2
x
level
6
4
2
0


2
1
0
−1
−2
y
−2
−1
0
1
2
x
level
6
4
2
0

Quiz 2 (graded) - 30 minutes.

Questions about principal component analysis, clustering, LDA, word embeddings. Five multiple choice questions and one short answer question.

Section 3

Logistic regression

Logistic regression

Recall that we have two outcomes, 0 and 1. Then let $p(X)$ be the estimated conditional probability $P(Y = 1|X)$. Logistic regression assumes that

$$\log \frac{p(X)}{1 - p(X)} = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

Or

$$p(X) = s(X^T \beta) = \frac{1}{1 + e^{-X^T \beta}} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p)}}$$

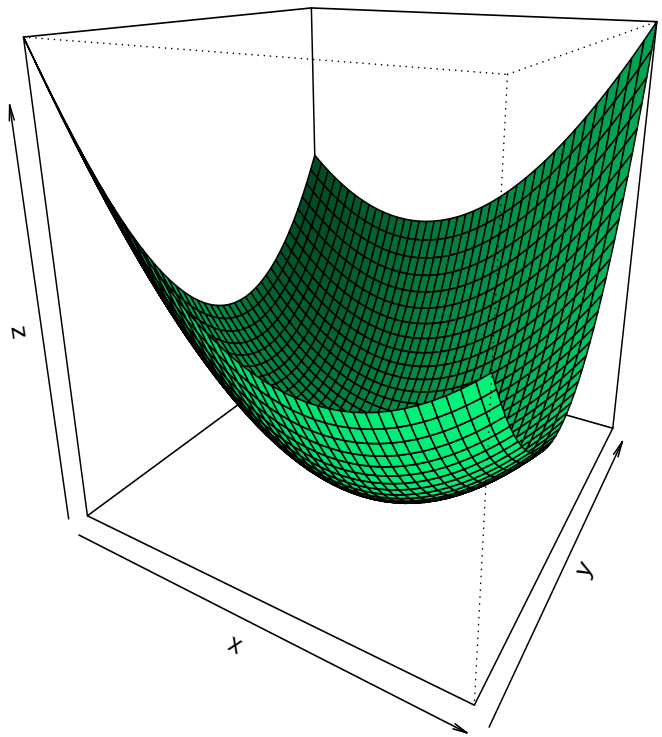
The binary cross-entropy *loss* function is

$$L(\beta) = -\sum_{i=1}^{N} \left(y^i \log p(x^i) + (1 - y^i) \log(1 - p(x^i)) \right)$$

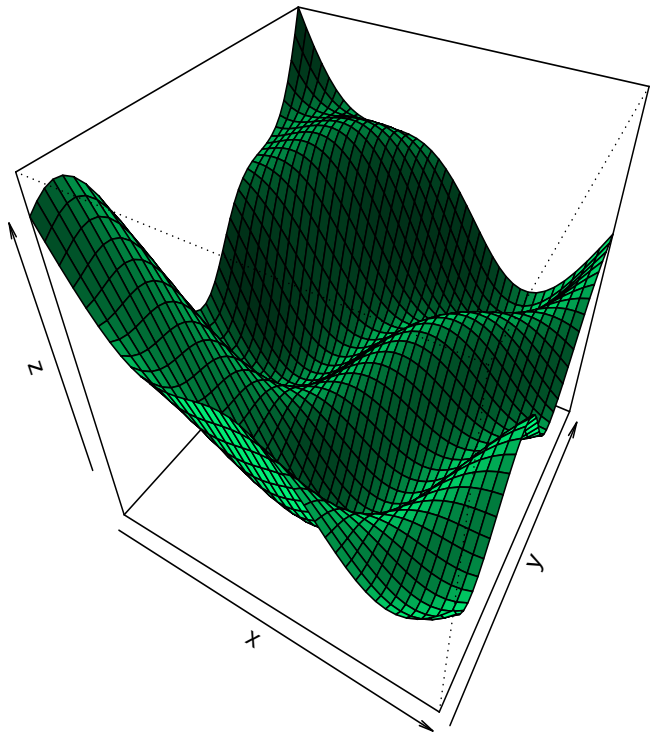
Gradient of the binary cross-entropy loss:

$$\frac{\partial}{\partial \beta^T} L(\beta) = -\mathbf{X}^T (\mathbf{y} - \widehat{\mathbf{y}})$$

Convex function



Non−convex function

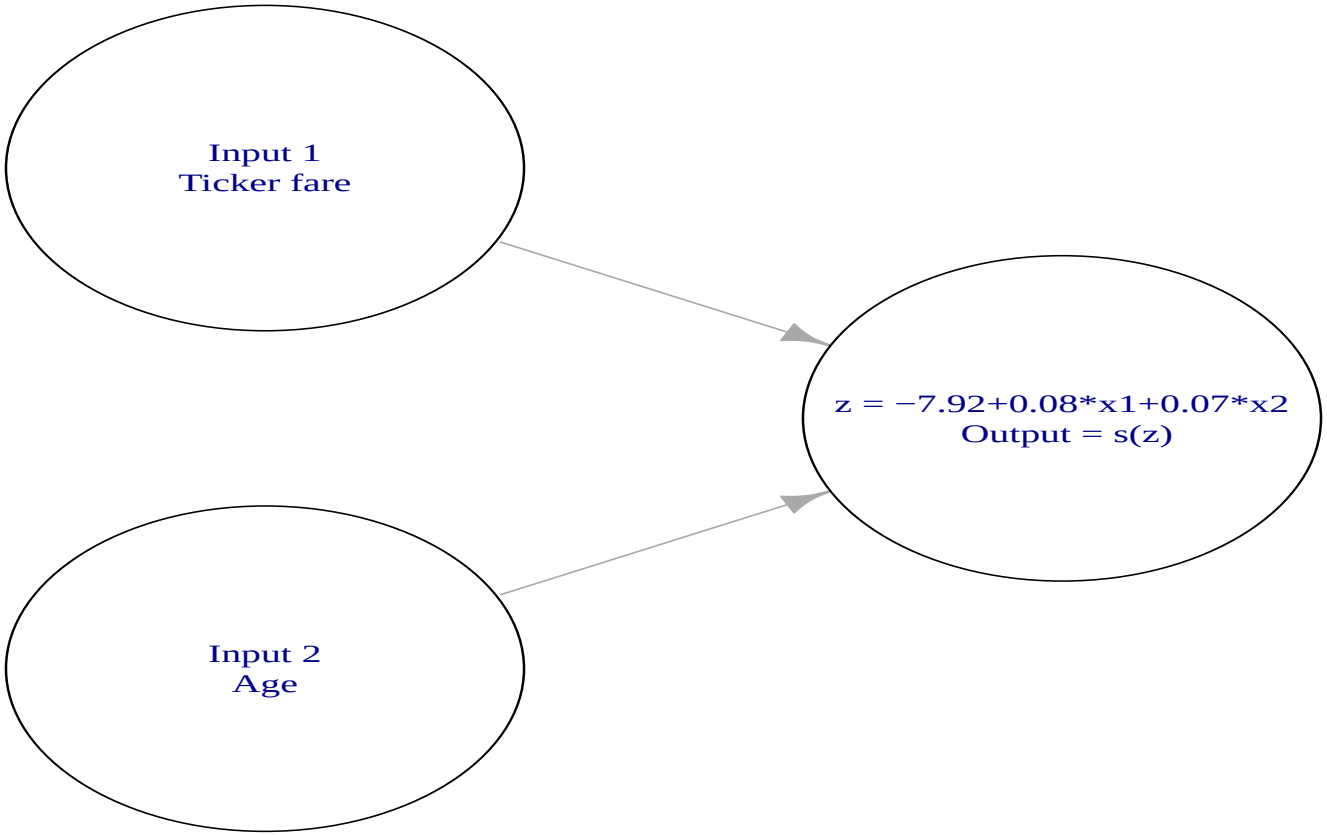


Logistic regression is trained by some version of gradient descent. The loss function is, in fact, convex. There is a unique point of minimum.

Section 4

Feedforward neural networks

Logistic regression as a neural network



The input vector is

$$X = (x_1, x_2)$$

The *neuron*:

- The vector of parameters is

$$\beta = (\beta_0, \beta_1, \beta_2) = (-7.92, 0.08, 0.07)$$

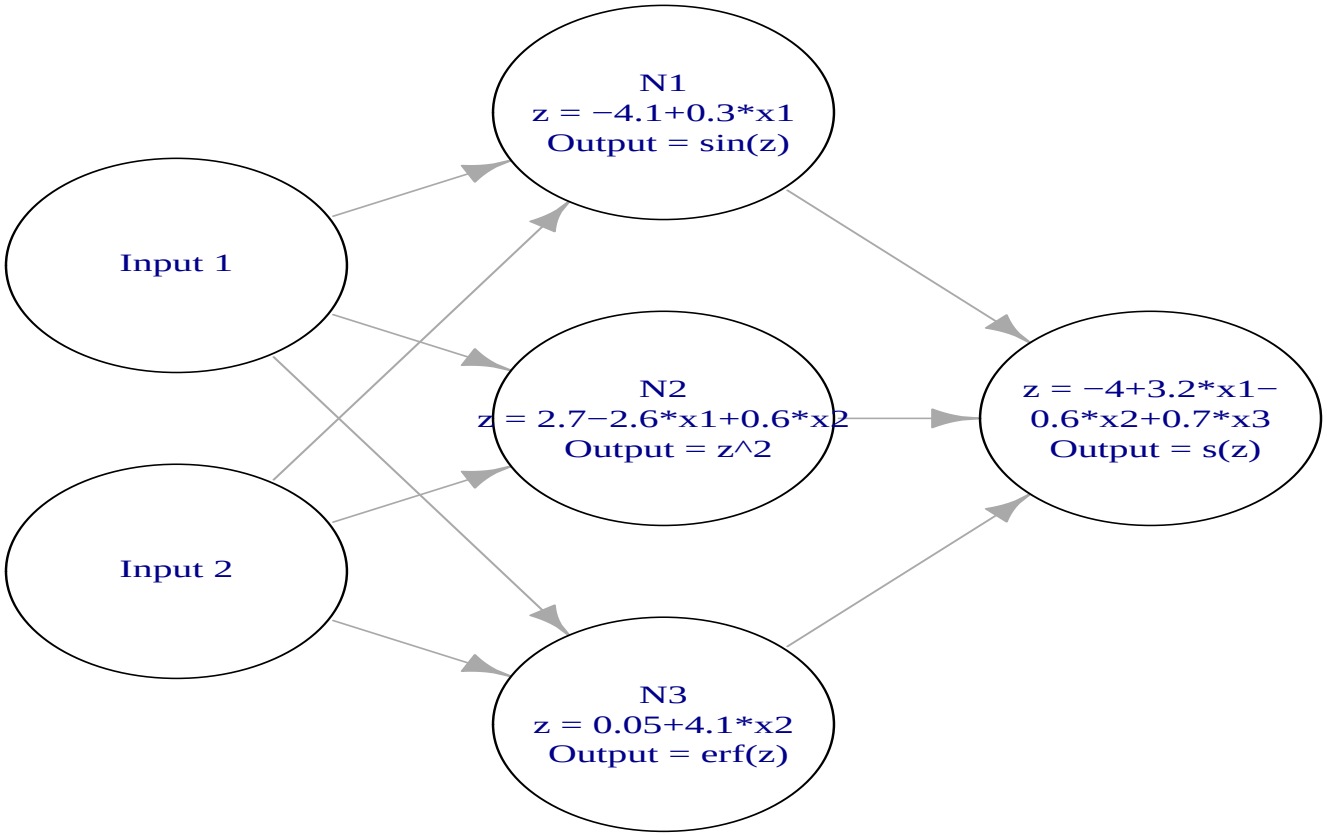
- The dot product is calculated as

$$z = (\beta_0, \beta_1, \beta_2) \cdot (1, x_1, x_2) = \begin{pmatrix} \beta_0 & \beta_1 & \beta_2 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ x_1 \\ x_2 \end{pmatrix} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

- The *activation function* is

$$s(z) = \frac{1}{1 + e^{-z}}$$

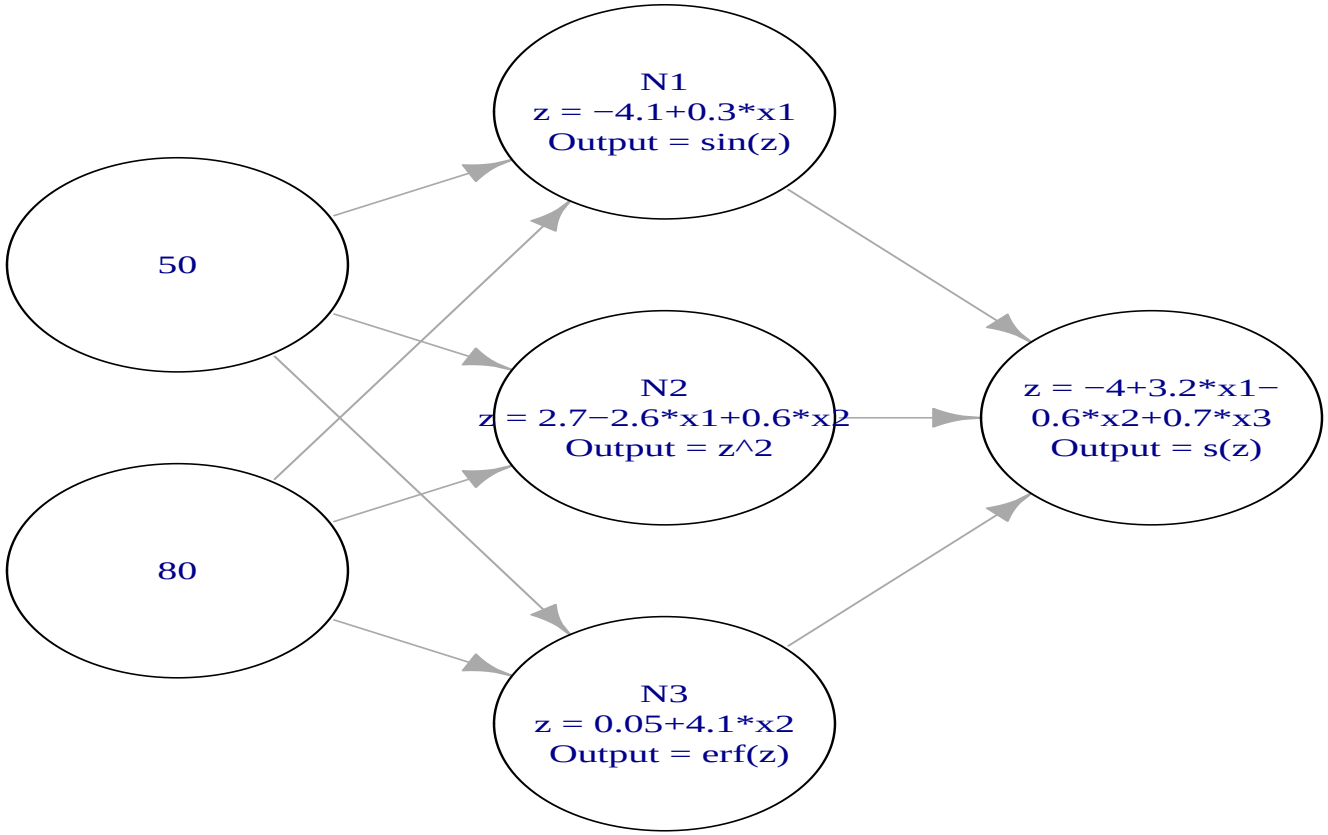
Hypothetical example

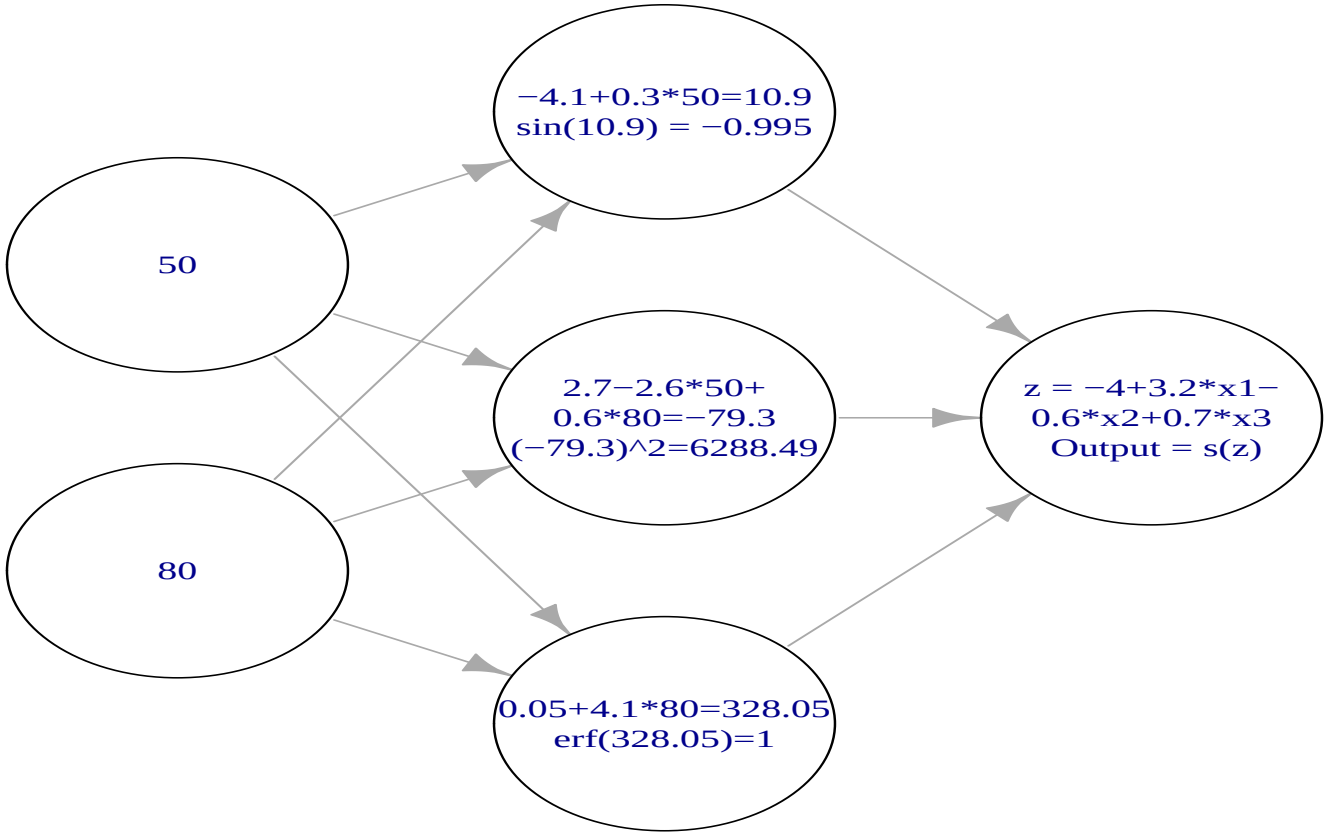


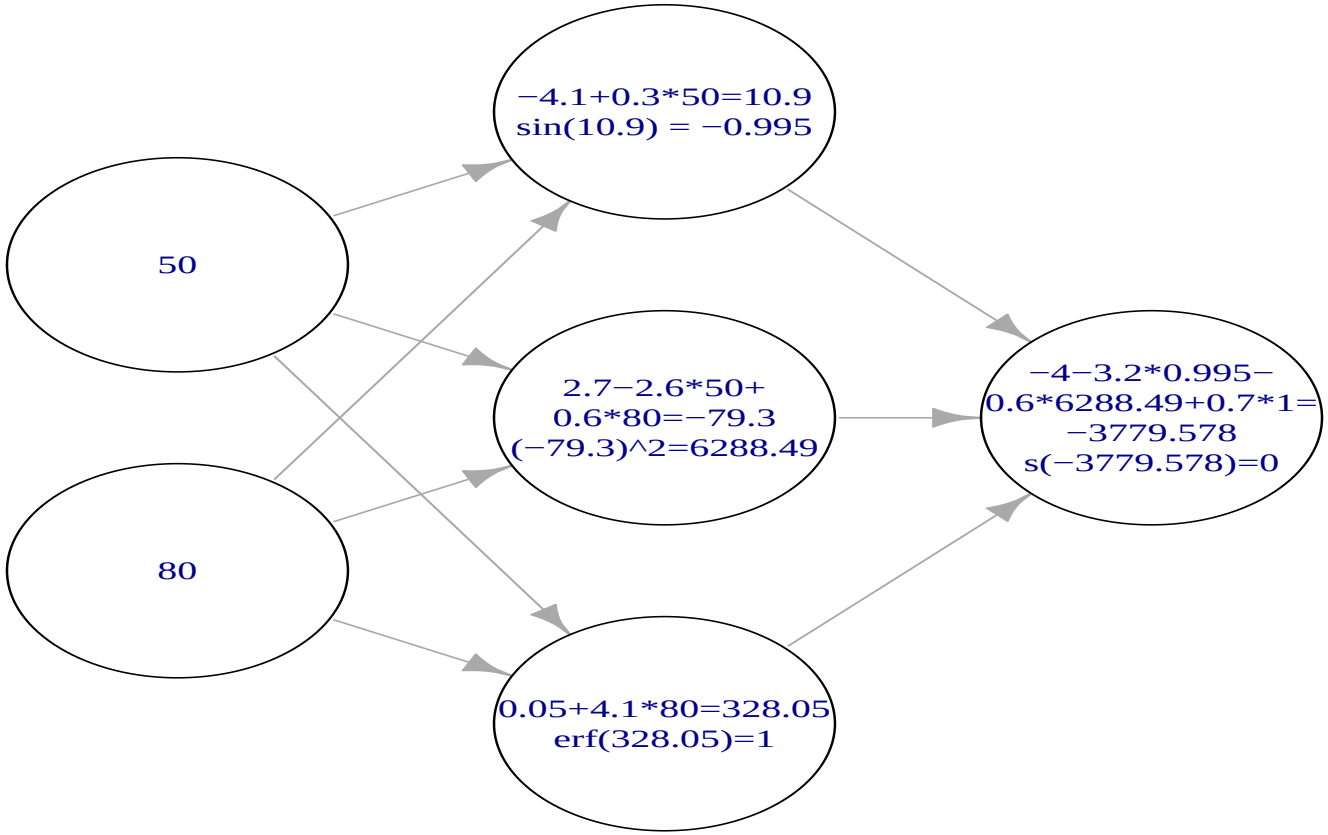
- Input 1 is x_1 and input 2 is x_2
- Hidden layer:

1) First neuron: activation function is $f(z) = \sin z$, weights are $\beta_0 = -4.1$, $\beta_1 = 0.3$, $\beta_2 = 0$.
2) Second neuron: activation function is $f(z) = z^2$, weights are $\beta_0 = -2.7$, $\beta_1 = -2.6$, $\beta_2 = 0.6$.
3) Third neuron: activation function is $f(z) = \mathrm{erf}(z)$, weights are $\beta_0 = 0.05$, $\beta_1 = 0$, $\beta_2 = 4.1$.

- Output layer: activation function is the sigmoid $f(z) = \frac{1}{1+e^{-z}}$, weights are $\beta_0 = -4$, $\beta_1 = 3.2$, $\beta_2 = 0.6$, $\beta_3 = 0.7$

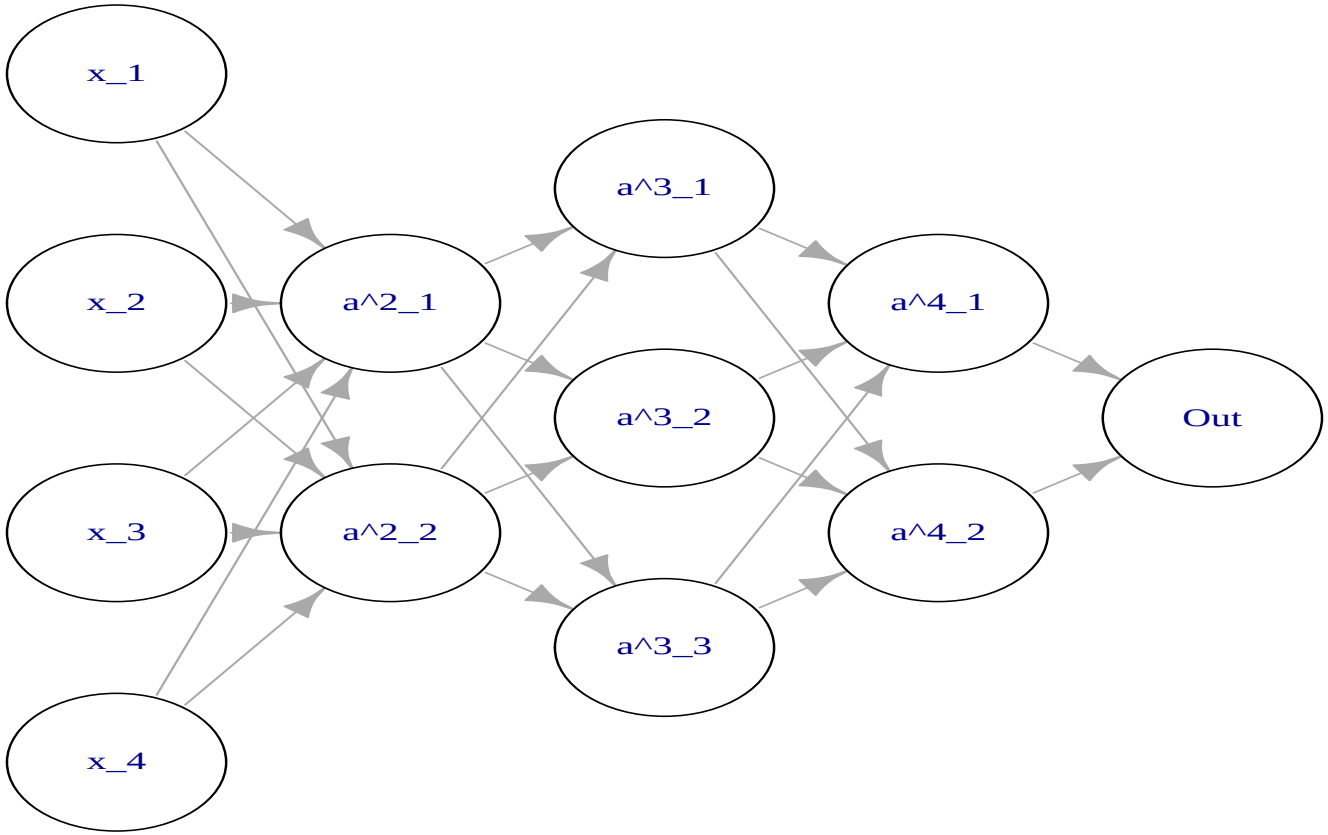

N1
z = −4.1+0.3*x1
Output = sin(z)
50
N2
z = 2.7−2.6*x1+0.6*x2
Output = z^2
z = −4+3.2*x1−
0.6*x2+0.7*x3
Output = s(z)
80
N3
z = 0.05+4.1*x2
Output = erf(z)


−4.1+0.3*50=10.9
sin(10.9) = −0.995
50
2.7−2.6*50+
0.6*80=−79.3
(−79.3)^2=6288.49
z = −4+3.2*x1−
0.6*x2+0.7*x3
Output = s(z)
80
0.05+4.1*80=328.05
erf(328.05)=1


−4.1+0.3*50=10.9
sin(10.9) = −0.995
50
2.7−2.6*50+
0.6*80=−79.3
(−79.3)^2=6288.49
−4−3.2*0.995−
0.6*6288.49+0.7*1=
−3779.578
s(−3779.578)=0
80
0.05+4.1*80=328.05
erf(328.05)=1

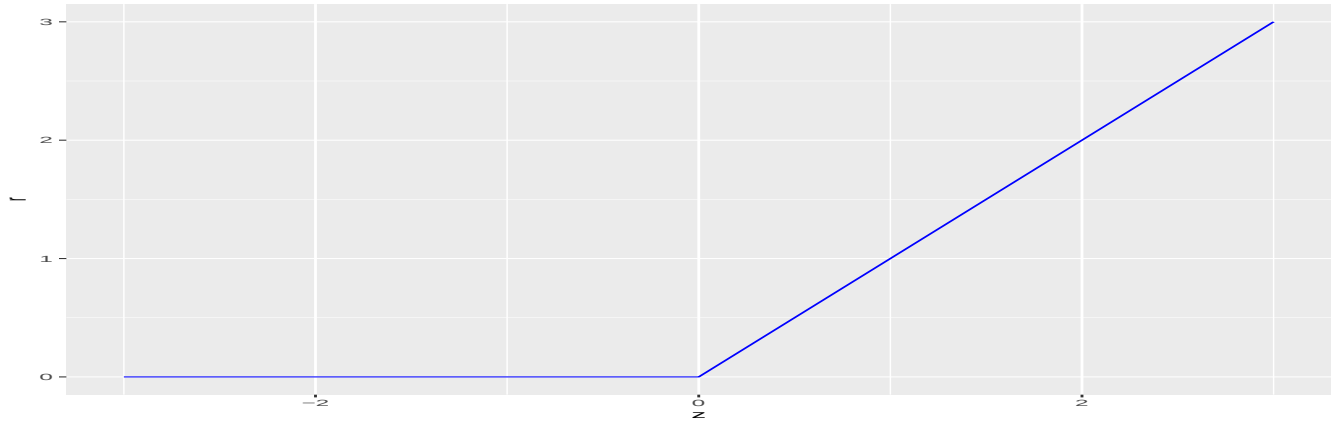
Fully connected architecture

Layer 1 - input, 3 hidden layers (of dimensions 2, 3, 2), Layer 5 - output

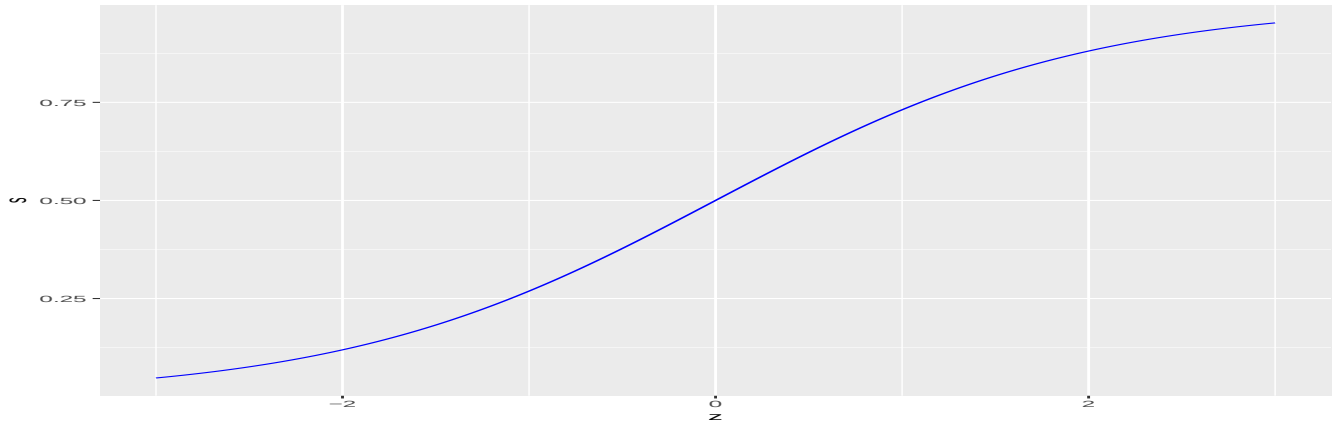


Activation functions (default choice for classification):

- Hidden layers: rectifier $f(z) = r(z) = \max(z, 0)$



- Output layer: $f(z) = s(z) = \frac{1}{1+e^{-z}}$



Layer 2 (first hidden layer):

$$\begin{aligned} a_1^2 &= r\left(b_1^1 + w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + w_{1,3}^1 x_3 + w_{1,4}^1 x_4\right) \\ a_2^2 &= r\left(b_2^1 + w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + w_{2,3}^1 x_3 + w_{2,4}^1 x_4\right) \end{aligned}$$

Layer 3:

$$\begin{aligned} a_1^3 &= r\left(b_1^2 + w_{1,1}^2 a_1^2 + w_{1,2}^2 a_2^2\right) \\ a_2^3 &= r\left(b_2^2 + w_{2,1}^2 a_1^2 + w_{2,2}^2 a_2^2\right) \\ a_3^3 &= r\left(b_3^2 + w_{3,1}^2 a_1^2 + w_{3,2}^2 a_2^2\right) \end{aligned}$$

Layer 4:

$$\begin{aligned} a_1^4 &= r\left(b_1^3 + w_{1,1}^3 a_1^3 + w_{1,2}^3 a_2^3 + w_{1,3}^3 a_3^3\right) \\ a_2^4 &= r\left(b_2^3 + w_{2,1}^3 a_1^3 + w_{2,2}^3 a_2^3 + w_{2,3}^3 a_3^3\right) \end{aligned}$$

Output:

$$a_1^5 = s\left(b_1^4 + w_{1,1}^4 a_1^4 + w_{1,2}^4 a_2^4\right)$$

Forward-propagation

Layer 2 (relu, 10 parameters):

1) matrix multiplication

$$z^2 = \begin{pmatrix} z_1^2 \\ z_2^2 \end{pmatrix} = \begin{pmatrix} b_1^1 & w_{1,1}^1 & w_{1,2}^1 & w_{1,3}^1 & w_{1,4}^1 \\ b_2^1 & w_{2,1}^1 & w_{2,2}^1 & w_{2,3}^1 & w_{2,4}^1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}$$

2) Activation (rectifier):

$$a^2 = \begin{pmatrix} a_1^2 \\ a_2^2 \end{pmatrix} = \begin{pmatrix} r(z_1^2) \\ r(z_2^2) \end{pmatrix}$$

Remark: in practice, layer dimensions can be large, but matrix multiplication and vectorized activation are very efficient operations.

Layer 3 (relu, 9 parameters):

1) matrix multiplication

$$z^3 = \begin{pmatrix} z_1^3 \\ z_2^3 \\ z_3^3 \end{pmatrix} = \begin{pmatrix} b_1^2 & w_{1,1}^2 & w_{1,2}^2 \\ b_2^2 & w_{2,1}^2 & w_{2,2}^2 \\ b_3^2 & w_{3,1}^2 & w_{3,2}^2 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ a_1^2 \\ a_2^2 \end{pmatrix}$$

2) Activation (rectifier):

$$a^3 = \begin{pmatrix} a_1^3 \\ a_2^3 \\ a_3^3 \end{pmatrix} = \begin{pmatrix} r(z_1^3) \\ r(z_2^3) \\ r(z_3^3) \end{pmatrix}$$

Layer 4 (relu, 8 parameters):

1) matrix multiplication

$$z^4 = \begin{pmatrix} z_1^4 \\ z_2^4 \end{pmatrix} = \begin{pmatrix} b_1^3 & w_{1,1}^3 & w_{1,2}^3 & w_{1,3}^3 \\ b_2^3 & w_{2,1}^3 & w_{2,2}^3 & w_{2,3}^3 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ a_1^3 \\ a_2^3 \\ a_3^3 \end{pmatrix}$$

2) Activation (rectifier):

$$a^4 = \begin{pmatrix} a_1^4 \\ a_2^4 \end{pmatrix} = \begin{pmatrix} r(z_1^4) \\ r(z_2^4) \end{pmatrix}$$

Layer 5 - output (3 parameters):

1) Matrix multiplication:

$$z^5 = \begin{pmatrix} b_1^4 & w_{1,1}^4 & w_{1,2}^4 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ a_1^4 \\ a_2^4 \end{pmatrix}$$

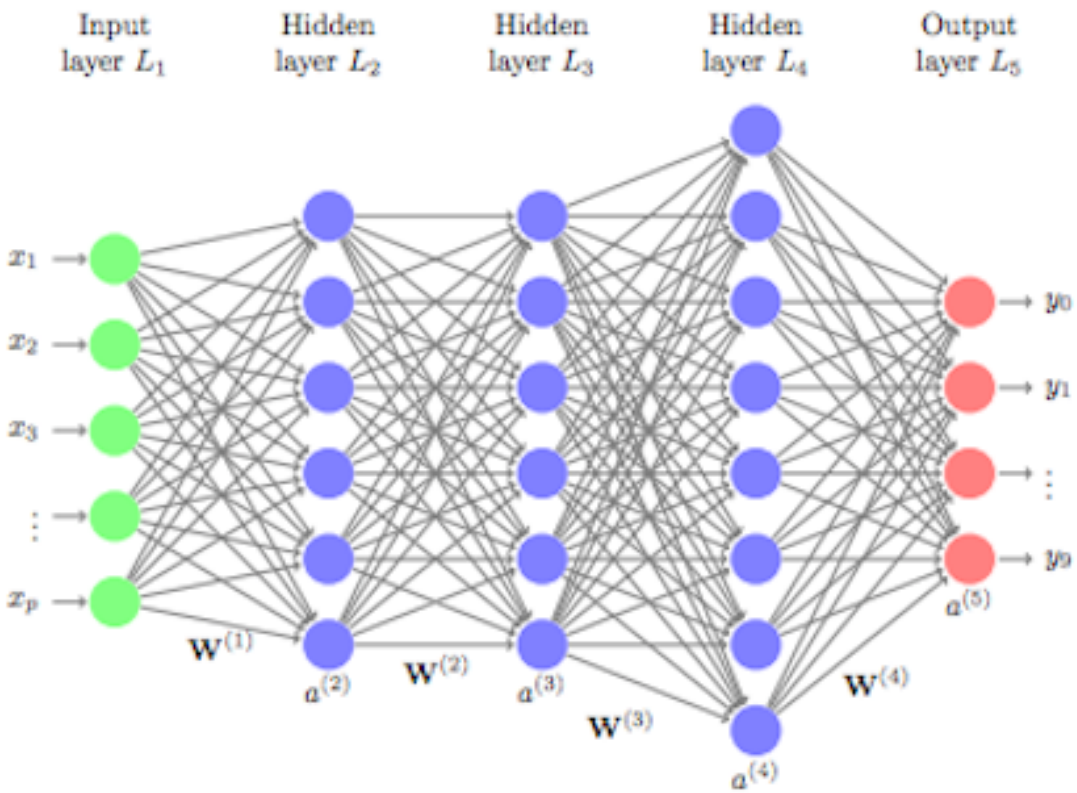
2) Activation (sigmoid):

$$a^5 = s(z^5) = \frac{1}{1 + e^{-z^5}}$$

The total number of parameters is $10 + 9 + 8 + 3 = 30$.

General description

A fully connected feed-forward artificial neural network consists of L layers, including layer 1 — the input layer, $L-2$ hidden layers, and layer L — the output layer. The number of units (neurons) in layer l is n_l for $l = 1, \ldots, L$.



Each unit has $n_{l-1}+1$ parameters, wher n_{l-1} is the dimension of the previous layer, and an activation function $f = f_{l-1}$ (same activation for all units in one layer). The output is

$$a_j^l = f(b_j^{l-1} + w_{j,1}^{l-1} a_1^{l-1} + \cdots + w_{j,k}^{l-1} a_k^{l-1})$$

For instance, $a^1 = x \in \mathbb{R}^p$, where $n_1 = p$ is the number of input variables.

Bias vector

$$b^{l-1} = \begin{bmatrix} b_1^{l-1} \\ \vdots \\ b_{n_l}^{l-1} \end{bmatrix}$$

Matrix of *weights*:

$$W^{l-1} = \left(w_{i,j}^{l-1}\right)_{i=1 j=1}^{n_l \quad n_{l-1}}$$

All parameters together:

$$\Theta = \{b^l, W^l\}_{l=1}^{L-1}$$

Section 5

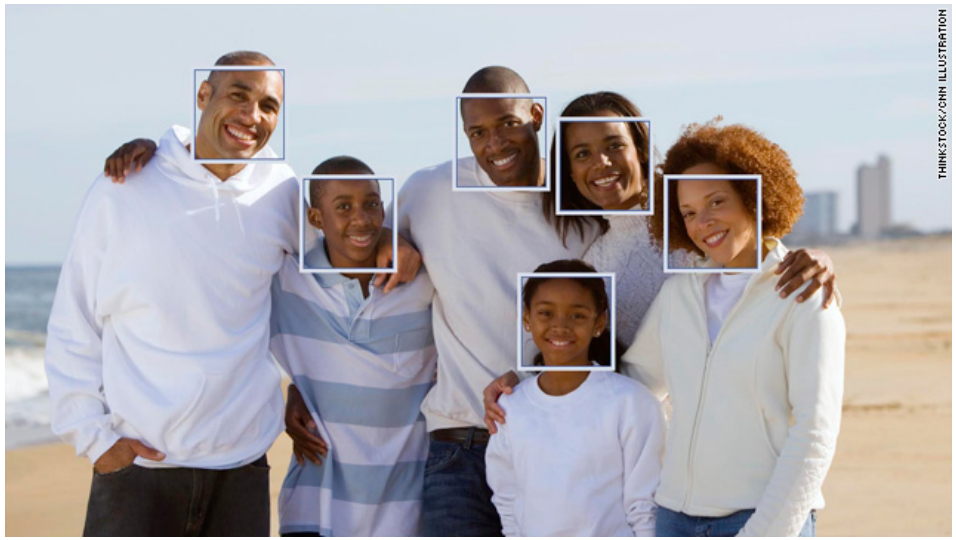
Training neural networks

Activation functions

Typically, we use the linear rectifier in hidden layers. There are alternatives available in "keras" — theoretically, we can even have different activation functions for different hidden layers, but the "default" choice is still the rectifier.

Activation for the output layer depends on the problem. We use the identity $\mathrm{id}(z) = z$ for regression, and the sigmoid $s(z)$ for binary classification.

Sometimes we have a multiple binary outcome, like for labelling faces on a photograph. Then the output layer has several sigmoid activation neurons.



For multiclass classification (for example, benign tumor vs malignant tumor vs no tumor in an X-Ray image), the output layer has a special neuron with a vector output.

Softmax activation:

$$\mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad \mathbf{z} = (z_1, \ldots, z_K) \in \mathbb{R}^K$$

Thus $h_\Theta(x^i)$ is a vector whose jth component is the probability that $\hat{y}^i$ belongs to class j.

Loss functions

Let Θ be the collection of all weights and biases of our network and let $h_\Theta(x)$ be the network's prediction on x. Some popular loss functions:

Binary cross-entropy:

$$J(\Theta) = \sum_{i=1}^{n} \left(-y^i \log h_\Theta(x^i) - (1 - y^i) \log(1 - h_\Theta(x^i))\right)$$

Mean squared error for regression:

$$J(\Theta) = \frac{1}{n} \sum_{i=1}^{n} \left(h_\Theta(x^i) - y^i\right)^2$$

Categorical cross-entropy (the sum is taken over classes and observations):

$$J(\theta) = -\sum_{j=1}^{M} \sum_{i=1}^{n} y_j^i \log h_\Theta(x^i)_j,$$

where y_j^i is 1 if y^i belongs to class j and 0 otherwise.

Gradient descent

Given a collection of parameters Θ, we can compute the output of a model $h_\Theta(x)$ for every x in the training set, and we have our loss function $J(\Theta)$.

For every parameter w (it may be a weight or a bias parameter), we compute the partial derivative

$$\frac{\partial J}{\partial w}$$

Together, all these partial derivatives form the gradient vector ∇J.

Parameters are initialized with some small random values.

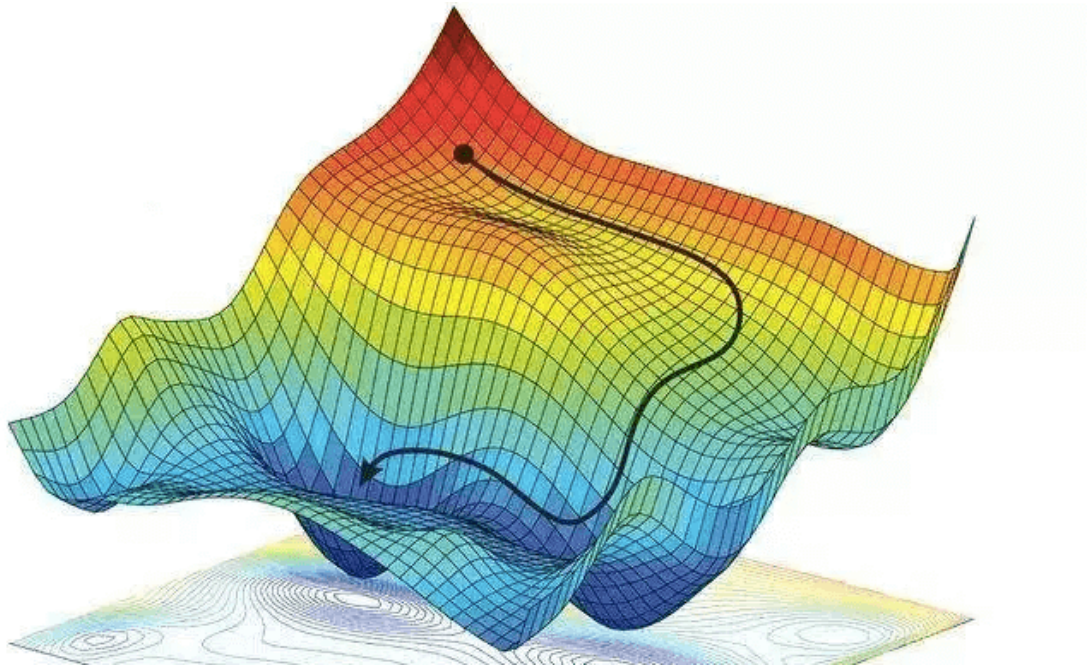
Update rule: $\Theta_{new} = \Theta_{old} - \alpha \cdot \nabla J(\Theta_{old})$.

The hyperparameter α is called the *learning rate*.

Usually, every update is done on a mini-batch rather than on the whole training set. This makes extensive use of matrix operations and the size of a mini-batch is a power of 2 — 32, 64, 128.

An *epoch* is one pass through the whole training set that typically has many mini-batch updates.

Variants of gradient descent: RMSProp, Adam, etc.



Performance metric

Model accuracy is measured by a suitable performance metric that may or may not be the same as the loss function.

For example, accuracy in classification problems is often measured by prediction accuracy, i.e., the fraction of correctly predicted cases. But the loss function is cross-entropy instead. Discuss why.

Similarly, for classification trees — to construct splits, we use cross-entropy, but we measure performance of the model by the overall accuracy.

Regularization

l_1 regularizer (with mean squared error loss):

$$J(\Theta) = \frac{1}{n}\sum_{i=1}^{n}\left(h_\Theta(x^i) - y^i\right)^2 + \lambda \sum_{l=2}^{L}\sum_{j=1}^{n_l}\sum_{j'=1}^{n_{l-1}}\left|w_{j,j'}^{l-1}\right|$$

l_2 regularizer:

$$J(\Theta) = \frac{1}{n}\sum_{i=1}^{n}\left(h_\Theta(x^i) - y^i\right)^2 + \frac{\lambda}{2} \sum_{l=2}^{L}\sum_{j=1}^{n_l}\sum_{j'=1}^{n_{l-1}}(w_{j,j'}^{l-1})^2,$$

where the sum of the weights is taken over all layers (biases not included). The parameter λ is called the *weight decay*.

Neuron dropout: in the training phase, ignore a random fraction p of nodes and corresponding activations. In the testing phase, use all activations, but reduce them by the factor p to account for missing neurons during training.

Activity: training an artificial neural network

Online version of Notebook 7 is HERE

Here you will be a part of risk evaluation team. We will predict bankruptcy of a company who has a credit in our bank by training an artificial neural network.

Source of data:
https://archive.ics.uci.edu/ml/datasets/Polish+companies+bankruptcy+data

Revision of Notebook 7 — WooClap

- https://app.wooclap.com/LTQJNU

Figure 1: Scan this QR code to open WooClal activity

Hyperparameters and settings

Parameters are updated through training via gradient descent.

Hyperparameters are predefined before training. This includes the number of layers, layer dimensions, regularization constants, learning rate etc.

Activation function / loss function / performance metric are defined by the problem.

Section 6

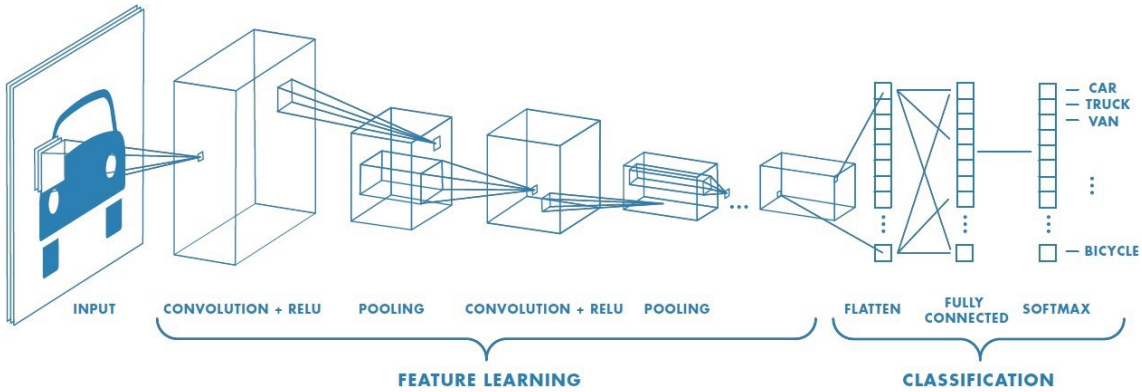
Conclusion

Types of networks

Fully connected network architecture is determined by the number of layers and the number of neurons in each layer.

Other popular types of artificial neural networks:

- Recurrent neural network for sequential data
- Transformers
- Convolutional neural network (image classification)
- Generative adversarial networks



Some remarks

A number of factors contribute to the boom of deep learning that happened in the last 10 years (main ideas in deep learning were invented in 1960s):

- Deep neural networks become effective when a lot of data are available.
- Improvements in algorithms (including seemingly simple ones) — usage of rectifier instead of sigmoid, Xavier initialization, Nesterov momentum for gradient descent. Various architectures of neural networks.
- Improvements in hardware. GPU and TPU.

Deep learning is a large and quickly evolving area of mathematics / computer science / engineering. If you are interested, take a special course on deep learning.

Section 7

Summary

Main takeaways

1. Constructing an artificial neural network is a long and complex process. It involves choosing the architecture (number of layers and layer dimensions), regularization method, learning rate, experimenting with hyperparameters and training by some form of gradient descent.

Concepts and ideas

1. Algorithm for supervised learning: feedforward neural network.
2. Parameters of a neural network: weights and biases.
3. Hyperparameters of network architecture: the number of layers and layer dimensions.
4. Hyperparameters of training: learning rate and regularization constants.
5. Activation function, loss function, optimizer, regularizer.
6. Batch gradient descent. Epoch.

At home

Notebook 8 - manual implementation of gradient descent.

A few theoretical questions about neural networks.

The online version of Notebook 8 is HERE